Understanding Qiskit Architecture

Quick Recap and Learning Objectives

A qubit stores quantum information, and quantum gates modify qubit states. Measurement converts quantum information into classical information, and quantum circuits are built by applying gates to qubits. Today's focus shifts away from quantum mechanics toward the software ecosystem that lets us program quantum computers. By the end, you will understand what Qiskit is, the architecture of the Qiskit ecosystem, how a program travels from Python code to a quantum computer, the difference between simulators and real hardware, and the role of the IBM Quantum Platform.

1. Why Do We Need Qiskit

Building a mobile application does not involve communicating with the processor using binary instructions directly. Instead, developers use programming languages, libraries, compilers, and debuggers, which simplify software development. Quantum computing works the same way. Instead of writing low-level instructions for a quantum processor, we use Qiskit, a software development kit that provides everything needed to design, optimise, execute, and analyse quantum programs. Without Qiskit, programming quantum computers would be extremely difficult.

What Is Qiskit

Qiskit is IBM's open-source Software Development Kit for quantum computing. It allows developers, researchers, and students to design quantum circuits, simulate quantum algorithms, execute programs on real quantum computers, and analyse and visualise quantum results. Think of Qiskit as the bridge between Python and a quantum computer.

2. The Qiskit Architecture

Every quantum program follows the same journey, passing through several layers that each have a specific responsibility.

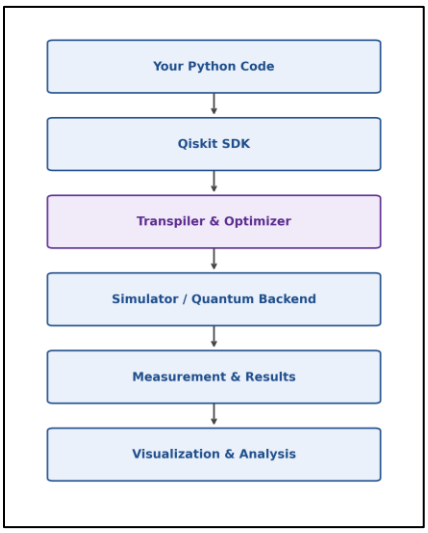


Figure 1: A quantum program's journey from Python code to visualised results.

3. Understanding Each Layer

Layer 1: Your Python Program

Everything begins with Python. This code describes what you want the quantum computer to do; it does not directly communicate with the hardware.

```
from qiskit import QuantumCircuit
qc = QuantumCircuit(1, 1)
qc.h(0)
qc.measure(0, 0)
```

Layer 2: Qiskit SDK

The SDK is the central software layer, acting as a toolbox containing everything required to build quantum applications. Its responsibilities include creating circuits, managing qubits, providing quantum gates, connecting to simulators, connecting to IBM Quantum hardware, and visualising results.

Layer 3: The Transpiler

Different quantum computers have different hardware layouts, and some gates may not be directly supported. The transpiler automatically prepares your circuit for the selected backend by optimising the circuit, reducing unnecessary gates, adapting it to the hardware's connectivity, and improving execution efficiency. This is similar to writing software for Windows and then running it on a Mac; some adjustments are necessary, and the transpiler performs that role for quantum hardware.

Layer 4: Backend

A backend is the system that executes your quantum circuit. A simulator reproduces quantum behaviour using a classical computer, and is fast, free, and ideal for learning and debugging. Real quantum hardware executes your circuit using physical qubits, giving true quantum computation and real hardware experience, but faces challenges such as noise, decoherence, queue times, and limited qubit availability. As a beginner, you will mostly work with simulators before moving to real hardware.

Layer 5: Results

Once execution completes, Qiskit returns measurement data, for example {'0': 503, '1': 497}. These results can then be visualised using charts and histograms, which is the final output of the quantum workflow.

4. Simulators vs Real Quantum Computers

Simulator	Real Quantum Computer
Runs on classical hardware	Uses physical qubits
Fast execution	Slower due to queue times
No hardware noise (ideal simulation)	Affected by quantum noise
Best for learning	Best for real experiments

Both are essential. Students generally develop and test programs using simulators before running them on actual quantum processors.

IBM Quantum Platform

The IBM Quantum Platform provides access to quantum simulators, real quantum computers, learning resources, the Circuit Composer, job management, and result visualisation. Instead of installing everything locally, you can access quantum hardware directly through your IBM Quantum account, making quantum computing accessible to anyone with an internet connection.

5. Common Beginner Mistakes

- "Qiskit is a quantum computer." In reality, Qiskit is a software development kit used to program quantum computers
- "Python directly controls quantum hardware." In reality, Python code is converted into quantum instructions before execution

- "A simulator is the same as a quantum computer." In reality, simulators imitate quantum behaviour, while real quantum computers execute operations on physical qubits and are affected by hardware limitations
- "The transpiler changes the algorithm." In reality, the transpiler preserves the algorithm's logic while rewriting the circuit into a form the selected hardware can execute efficiently

Summary: What to Remember

- Qiskit is IBM's open-source SDK for quantum computing
- Every quantum program follows a software workflow from Python to hardware
- The transpiler prepares circuits for specific quantum devices without changing their logic
- A backend, either a simulator or real hardware, is where the circuit is executed
- Results are returned as measurement counts and can be visualised using Qiskit tools
- The IBM Quantum Platform gives access to simulators, real hardware, and learning resources over the internet